Step 1 — Connect to One MCP Server and List Its Tools — Hints

> Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

You're building the connection layer and nothing else — no model, no loop. A client launches the server as a subprocess over stdio, does the MCP handshake, and asks what tools it has.

The exact pieces to look up and use (all from Doc06's MCP client example):

- `from mcp import ClientSession, StdioServerParameters` and `from mcp.client.stdio import stdio_client`.
- `StdioServerParameters(command="python", args=["servers/notes_server.py"])` — how to launch the server.
- `async with stdio_client(params) as (read, write):` then `async with ClientSession(read, write) as session:` then `await session.initialize()` — the required handshake.
- `await session.list_tools()` returns a `ListToolsResult` with a `.tools` list; each tool has `.name`, `.description`, and `.input_schema` (snake_case — look at it closely).
- For the example server: `from mcp.server.mcpserver import MCPServer`, `@mcp.tool()`, `mcp.run(transport="stdio")` (Doc06 / Project 7's shape).

Keep one place — `MCPServerPool` — that connects and stores what it discovers. Even with one server, give each tool a namespaced name `servername__toolname` and a routing entry now; Step 3 is where you'll be glad it's already there.

Hint 2 — The plan, and almost the whole thing

```
config.py / exceptions.py / utils/logger.py:
    the Doc01 trio — logger goes to STDERR (stdout is the MCP channel)

servers/notes_server.py:
```

```
    MCPServer("notes-server"), two @mcp.tool() functions, mcp.run("stdio")

server_pool.py:
    class MCPServerPool(servers):
        sessions = {}; routing = {}; named_tools = []
        stack = AsyncExitStack()
        async connect_all():
          for each server:
            rd, wr  = await stack.enter_async_context(stdio_client(params))
            session = await stack.enter_async_context(ClientSession(rd, wr))
            await session.initialize()
            sessions[name] = session
            for tool in (await session.list_tools()).tools:
                namespaced = name + "__" + tool.name
                routing[namespaced] = (name, tool.name)
                named_tools.append((namespaced, tool))
        async aclose(): await stack.aclose()

main.py:
    asyncio.run: connect_all(), then print each tool's name + inputs
```

Two things people get wrong here, both worth checking by running:

1. It's `tool.input_schema`, **not** `tool.inputSchema` — the camelCase attribute raises `AttributeError` in the current SDK.
2. Store the `AsyncExitStack` on the pool and enter every context through it, so one `aclose()` tears everything down. (Step 4 adds the timeout; don't add it yet.)

The whole file set, runnable, is in the Solution.